CS456 – Modern Cybersecurity

Mini Group Project 1: Resilient Security Systems – Part 1 (Failure Detection)

Objective

In our class so far, we have primarily focused on Confidentiality and Integrity. However, in real-world cybersecurity, **Availability** is a core pillar of the CIA triad alongside Confidentiality and Integrity. Availability guarantees that authorized entities have timely, uninterrupted access to critical information and operational services. Security-critical services, such as Authentication Servers (Kerberos KDCs), Intrusion Detection Systems (IDS), Certificate Authorities (CAs), and Security Information and Event Management (SIEM) pipelines, are high-value targets for Denial of Service (DoS) attacks, memory corruption exploits, resource starvation, and unexpected process crashes.

To maintain high availability, security engineers deploy **Availability Tactics**: software architectural mechanisms specifically designed to **detect faults, withstand disruptions, and recover operations**. This two-week mini group project will challenge your team to build a resilient, distributed security service.

Background

In mission-critical infrastructure, an outage can be as devastating as a data breach. Threat actors frequently attempt to eliminate availability through:

- **Volumetric Denial of Service (DoS):** Flooding network pipes or daemon queues to starve resources (e.g., Mirai botnets).
- **Exploitative Crashes:** Triggering unhandled exceptions, buffer over-reads, null-pointer dereferences, or memory exhaustions to terminate critical services remotely.
- **Ransomware & Hostage Scenarios:** Sabotaging services to hold functionality hostage (e.g., WannaCry).

To defend availability, systems must not assume components are infallible. Instead, they must be designed with *Architectural Availability Tactics*, reusable design decisions specifically intended to handle faults, mitigate disruptions, and maintain uptime. Architectural tactics for availability generally fall into three operational categories: **Detecting Faults**, **Recovering from Faults**, and **Preventing Faults**.

- **Detecting Faults:** Before a system can react to a crash or disruption, it must reliably detect that an anomaly has occurred.

    - **Heartbeat/Ping-Echo:** Periodic liveness signals exchanged between a monitor and a monitored service. If the monitor fails to receive a beat within an established timeout threshold, it declares the service unresponsive or crashed.
    - **Watchdog Timer:** A dedicated hardware or software countdown timer that must be periodically reset ("kicked") by the critical process. If a thread hangs in an infinite loop or deadlocks due to malicious input, it fails to kick the timer; when the timer hits zero, it triggers a reboot, restart, or alert.
    - **Sanity Checking and Condition Monitoring:** Continuously validating the internal operational boundaries, checksums, and invariants of a running service. Flags buffer threshold anomalies, memory corruption early signs, or out-of-order sequence numbers before full process termination.
- **Recovering from Faults:** Once a fault is detected, the system executes tactics to preserve operational continuity or restore normal functioning.
    - **Active Redundancy:** Multiple identical replicas process incoming requests simultaneously in real time. All replicas receive the same input and synchronize operations. If the primary node crashes, the secondary node serves requests with zero failover downtime.
    - **Passive Redundancy:** A primary node processes requests while periodically snapshotting its current operational state to a shared store or backup node (checkpointing). When the monitor detects a primary node crash, the standby backup spins up, loads the latest checkpoint, and resumes service.
    - **Rollback/State Resynchronization:** Reverting the system to a known good, uncorrupted previous state (via transaction logs or checkpoints) following an unexpected crash or state corruption.
    - **Degraded Operation:** Prioritizing core mission-critical functionality while selectively disabling non-essential features under extreme load or component failure. If a log-indexing pipeline becomes backlogged during a volumetric flood, an API might switch to local disk append-only logging to maintain availability for authentication requests.
- **Preventing Faults:** Tactics aimed at stopping potential faults from turning into active system failures.
    - **Removal from Service (Proactive Eviction):** Temporarily taking components offline for rejuvenation, memory cleanups, or garbage-collection flushes before fragmentation causes an unexpected crash.
    - **Process Isolation & Sandboxing:** Partitioning untrusted or resource-heavy operations into independent processes or restricted security contexts (e.g., containers, separate OS processes).
    - **Rate Limiting/Shedding Load:** Restricting request intake to prevent thread-pool exhaustion and volumetric resource starvation.

The Heartbeat Tactic: A **Heartbeat** is a fault-detection mechanism used in distributed and high-availability systems. An independent monitor periodically exchanges liveness messages with the monitored service across a communication channel. Heartbeat protocols generally fall into two models:

- **Push-Based Heartbeat (Unidirectional Notification):** The critical process autonomously emits a small datagram (a "beat") at a regular, fixed period T to the monitor. The monitor maintains an expiring countdown timer or sliding window. Each arrival of a valid beat resets the timer. If N consecutive periods elapse without a beat, the monitor flags a timeout and raises a fault alert.
- **Pull-Based / Ping-Echo Heartbeat (Bidirectional Query):** The monitor initiates the check by transmitting a PING query to the critical process at interval T. The critical process must reply with an ACK or ECHO within a defined response window W. If no response returns within W, or after N missed replies, the monitor triggers an alert.

Engineering Trade-offs in Heartbeat Design: When implementing a heartbeat protocol, system designers must calibrate three key parameters:

- **Frequency (T):** Sending beats too frequently (e.g., every 5 ms) consumes CPU cycles and saturates local network sockets. Sending beats too slowly (e.g., every 60 seconds) prolongs the system's "mean time to detect" (MTTD), leaving the service silently offline for a full minute before anyone notices.
- **Timeout Threshold (N or Window W):** Transient network latency or brief thread locks can delay a packet. Setting a threshold too aggressively results in **false positives** (flagging a healthy process as dead). Setting it too loosely delays recovery.
- **Thread/Process Concurrency:** Heartbeat emission must be decoupled from the primary service logic. If a service processes long-running authentication requests synchronously on its main thread, it could fail to emit a heartbeat simply because it is busy, triggering an erroneous failover.

Cybersecurity Perspective: Securing the Heartbeat

In an adversarial environment, an unmonitored or unauthenticated heartbeat channel can itself become an attack vector:

- **Heartbeat Spoofing:** If an attacker compromises or disables a critical security daemon (like an IDS sensor), they could forge heartbeat packets to the monitor, tricking the monitoring infrastructure into reporting that all security controls are operational ("green") while an intrusion continues undetected.
- **Denial of Service on Monitoring:** Flooding the monitor's listening socket can cause it to drop valid beats, falsely triggering failovers or alarms.
- **Tampering & Replay:** Stale heartbeat messages captured off the wire could be replayed by an attacker to mask a deliberate process crash.

Designing a secure availability tactic requires considering not just system crashes, but also how an adversary might manipulate the detection channel itself.

Project Instructions

For this mini group project, you will create a lightweight security service protocol and simulate a realistic, non-deterministic system failure/crash. In Part 1, you will design and implement a monitoring agent that reliably distinguishes between healthy operations and process crashes (fault detection).

You may use any languages or libraries.

Task 1: Team Selection

You will work in teams of 3-4. Form a team with your classmates. You may use the Canvas discussion board to find teammates.

Task 2: Select a Security Domain

Choose a lightweight security service for your prototype. Keep the functional logic minimal (a basic prototype is expected); the focus is on the monitoring mechanism. Examples include:

- **Authentication/Ticket Service:** Validates dummy credentials or issues session tokens.
- **Network Sentinel/Firewall Agent:** Inspects dummy packet payloads and flags blacklisted IPs.
- **Cryptographic Key-Storage Daemon:** Responds to requests for public certificates or encrypted session wrappers.
- **Integrity Monitor/Log Aggregator:** Collects and hashes simulated event logs.

Task 3: Implement the Critical Process with Simulated Failure

Implement the core service as an independent running process. The process must experience unpredictable crashes during normal execution.

Rule: Do not hard-code an arbitrary static conditional (e.g., if (request_count == 5) exit()). Implement stochastic or environmental crash triggers, such as:

- A random probability trigger on incoming requests (e.g., simulating a memory leak or segfault on malformed input).
- A randomized background fault-injection thread.

The failure *must terminate/exit the process completely* (do not simply put the process to sleep).

Task 4: Implement the Heartbeat Monitor

Implement an independent **Monitor Process** that runs separately from the critical service (separate processes/sockets). The monitored service and the monitor must communicate periodic liveness signals:

- *Option A (Push/Periodic Heartbeat):* The critical service sends a "HEARTBEAT" datagram to the monitor every T milliseconds.
- *Option B (Pull/Ping-Ack):* The monitor pings the service every T milliseconds, requiring a timely response.

Fault Detection Logic:

- Define an explicit timeout threshold (e.g., missing N consecutive beats or exceeding a sliding window timeout).
- When the critical service crashes, the monitor must detect the failure, print an alert to standard output with timestamps, and log the incident.
- The monitor must not crash when the monitored service dies.

Task 5: Adversarial Assessment

At least one team member should take on the role of an adversarial auditor to analyze and test vulnerabilities in the heartbeat channel:

- Attack 1 – Heartbeat Spoofing: *Can an external script transmit fake heartbeat messages to the monitor so that the monitor reports the service is healthy even after the critical process has crashed?*
- Attack 2 – Replay Attack/Tampering: *If the heartbeat payload includes status data or a timestamp, can an attacker capture a valid packet and replay it indefinitely to mask downtime?*

Reflection (individual)

Take a few minutes to reflect on your personal workflow, the concepts you encountered, and the tools you used to complete this assignment.

1. How difficult was this assignment on a scale of 1 (very easy) – 5 (very challenging)?
2. What was easy about this assignment and why?
3. What was challenging about this assignment and why?
4. What external tools or resources did you use to complete this assignment and where were they helpful?

Deliverables

Please submit the following via Canvas:

1. Each team must submit a single repository link (e.g., GitHub/GitLab) containing the following:

a. **Critical Service Process:** Clean, documented source code for the lightweight security service featuring the non-deterministic crash injection mechanism.
b. **Heartbeat Monitor Process:** Source code for the independent watchdog agent that manages timers/windows, processes incoming heartbeats, and triggers explicit failure alerts upon a crash.
c. **Adversarial Audit Script:** A standalone script or utility designed to attempt heartbeat spoofing, message replay, or false liveness injection against the monitor.
d. **Role Distribution:** A separate CONTRIBUTIONS.md listing each team member's specific contributions to the service, monitor, security audit, and video demo.

2. Each team must submit a single PDF report containing:
    a. **Names of all teammates:** First and last.
    b. **Service Overview:** A concise description of your team's chosen security domain and the simulated failure scenario.
    c. **Execution & Setup Instructions:** Unambiguous, step-by-step terminal commands to compile, configure, and execute the service, monitor, and attack script.
    d. Architectural Parameter Justification:
        i. Heartbeat pattern (Push vs. Pull/Ping-Echo).
        ii. Timing calibration: Values chosen for heartbeat interval (T), timeout window (W), and missed-beat threshold (N), justifying how your team balanced detection speed (MTTD) against false positives.
    e. **Adversarial Audit Summary:** 1-2 paragraphs detailing the vulnerability of the baseline heartbeat, and whether or not the attack script exploited it successfully and why.
3. Each team must submit a single video demonstration that meets the following criteria:
    a. Time Limit: Strictly under 3 minutes (anything over will not be reviewed). If Canvas doesn’t allow the video upload, please provide a Google Drive or unlisted YouTube link.
    b. Show both processes starting up and the monitor acknowledging valid heartbeats in real time.
    c. Show the critical service triggering its non-deterministic crash (you may need to edit the video to show it, since there’s a time limit on the video but the crash is non-deterministic).
    d. Show the attacker script executing against the monitor (demonstrating either the exploit succeeding or failing on the build).
    e. Voiceover: Explain what is going on.

4. Each individual (separately in Canvas) must submit their reflection with the four reflection questions.

Note: For team deliverables, one person may submit. For individual reflections, each individual must submit.

Grading

Here is the grading scheme for the project report:

- Service script (15)
- Heartbeat script (15)
- Adversarial script (15)
- PDF report (20)
- Video demo (20)
- Individual reflection (15)

TOTAL: 100 points